

MLOps Lab 1

Experiment Tracking and Model Registry with MLflow

Case Study: CartVista Customer Churn Prediction

Student Name: Priyanshu Tripathi

Student ID: 2025ae05230

Course: AIML ZG523 – MLOps

This submission documents the required Lab 1 activities, experiment tracking results, model registration, consumer execution, and written exercises.

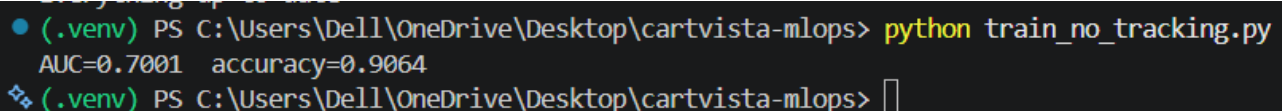
1. Step 2 Checkpoint – Accuracy vs AUC

The model achieved high accuracy (0.9064) while its AUC was 0.7001 because the churn dataset is imbalanced: most customers do not churn. A model can obtain high accuracy by predicting the majority class (no churn) for most customers, while still being only moderately effective at distinguishing customers who will churn from those who will not. AUC is therefore more informative than accuracy for evaluating the ranking/discrimination ability of the model in this imbalanced setting.

Observed result from the submitted run: AUC = 0.7001, Accuracy = 0.9064.

2. MLflow Experiment Tracking

The MLflow experiment 'cartvista-churn' contains the tracked training runs. The comparison view records model parameters, data fingerprint, and evaluation metrics, allowing candidate models to be compared using the same data snapshot.

A terminal window with a dark background. The prompt is a blue dot followed by (.venv) PS. The command is python train_no_tracking.py. The output is AUC=0.7001 accuracy=0.9064. The prompt is repeated on the next line.

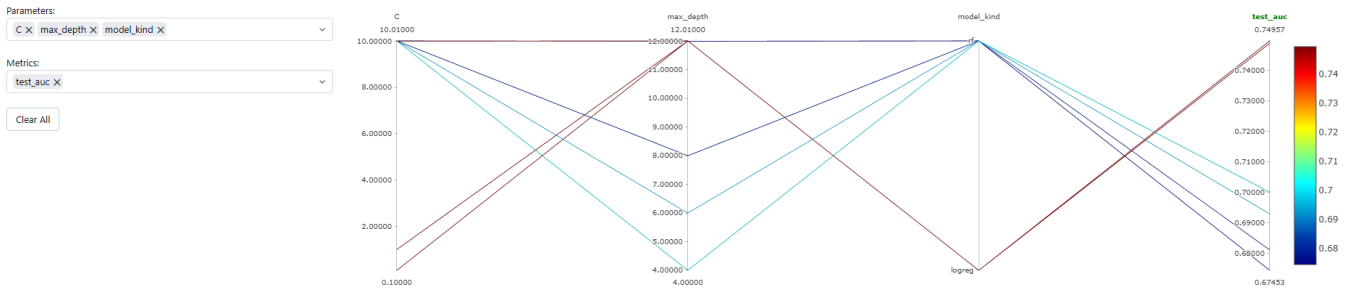
```
(.venv) PS C:\Users\Dell\OneDrive\Desktop\cartvista-mlops> python train_no_tracking.py
AUC=0.7001 accuracy=0.9064
(.venv) PS C:\Users\Dell\OneDrive\Desktop\cartvista-mlops> 
```

Figure 1. MLflow comparison view showing tracked runs, parameters and test AUC.

Comparing 13 Runs from 1 Experiment

Visualizations

Parallel Coordinates Plot Scatter Plot Box Plot Contour Plot



Run details

Figure 2. MLflow parameter and metric comparison details.

3. E1 – Precision and Recall

For CartVista, precision is particularly important because retention offers cost money to send. Precision measures how many customers identified as likely to churn actually churn. A low precision would cause the retention team to spend money on many customers who would not have churned anyway. Recall is also important because missing true churners can result in lost customers. However, given the explicit cost of sending offers, the team should place strong emphasis on precision while considering recall as a complementary metric and selecting an appropriate operating threshold.

Required implementation: Add precision and recall to train_tracked.py and rerun the best configuration.

4. E2 – Third Challenger and Model Registry

The lab requires adding GradientBoostingClassifier as a third model family, exposing learning_rate and n_estimators, running two configurations, and rerunning register_model.py. The registry maintains immutable model versions while the @champion alias points to the current champion model.

Parameters

Show diff only

C	0.1	1.0	10.0	0.1	1.0
data_fingerprint	fb2752fe23	fb2752fe23	fb2752fe23	fb2752fe23	fb2752fe23
max_depth	6		4	8	12
model_kind	rf	logreg	logreg	rf	logreg
n_estimators	100		50	200	400

Metrics

Show diff only

test_accuracy	0.906	0.908	0.908	0.908	0.906	0.906	0.906	0.908	0.908
test_auc	0.693	0.749	0.75	0.75	0.7	0.681	0.675	0.749	0.75
test_f1	0.017	0.05	0.05	0.05	0	0.033	0.092	0.05	0.05

Figure 3. MLflow experiment table showing the tracked CartVista runs.

Experiments

Search Experiments

Default

cartvista-churn

cartvista-churn

Provide Feedback

Add Description

Share

Q metrics.mse < 1 and params.model = "tree"

Time created

State: Active

Datasets

Sort: Created

Columns

Group by

New run

Table

Chart

Evaluation

Experimental

Traces

Experimental

Run Name	Created	Dataset	Duration	Source	Models
rf-run	11 hours ago	-	9.8s	train_tracked.py	sklearn
logreg-run	11 hours ago	-	4.3s	train_tracked.py	sklearn
logreg-run	11 hours ago	-	4.4s	train_tracked.py	cartvista-churn V2
logreg-run	11 hours ago	-	6.3s	train_tracked.py	sklearn
rf-run	11 hours ago	-	5.6s	train_tracked.py	sklearn
rf-run	11 hours ago	-	6.2s	train_tracked.py	sklearn
rf-run	11 hours ago	-	5.6s	train_tracked.py	sklearn
logreg-run	5 minutes ago	-	9.7s	train_tracked.py	sklearn
logreg-run	5 minutes ago	-	5.6s	train_tracked.py	sklearn
logreg-run	4 minutes ago	-	4.7s	train_tracked.py	sklearn
rf-run	4 minutes ago	-	5.8s	train_tracked.py	sklearn
rf-run	4 minutes ago	-	5.7s	train_tracked.py	sklearn
rf-run	4 minutes ago	-	7.0s	train_tracked.py	sklearn

Show more columns (11 total)

Figure 4. MLflow comparison visualization with model parameters and test AUC.

5. E3 – Tags for Humans

Tags are appropriate for author and purpose because they describe contextual metadata about a run rather than the configuration used to train the model. Parameters such as `C`, `n_estimators` and `max_depth` define the model configuration and can affect the resulting model. An author or purpose is descriptive metadata used to organize, filter and understand experiments. Therefore, tags are the better home for this information.

Required implementation: Add an author tag using 'Priyanshu Tripathi' and a purpose tag to each run, then filter the runs in the MLflow UI.

6. E4 – Data Fingerprint

The `data_fingerprint` is an MD5-based identifier recorded with each run. If one row is removed from `churn_data.csv` and another experiment is executed, its fingerprint should differ from earlier runs. This prevents a disagreement such as Priya and Arjun claiming that they evaluated their models on the same dataset when one of them actually used a different data snapshot. The original dataset should be restored after the experiment.

7. Model Registration and Champion

The submitted terminal evidence shows that `register_model.py` selected a best tracked run with AUC 0.7496, created a new registered model version, and set the `@champion` alias. The registry output shown in the submitted evidence reports creation of version 3 of `cartvista-churn` and assignment of `@champion` to that version.

```
(.venv) PS C:\Users\Dell\OneDrive\Desktop\cartvista-mlops> python register_model.py
C:\Users\Dell\OneDrive\Desktop\cartvista-mlops\.venv\Lib\site-packages\mlflow\utils\requirements_utils.py:20: UserWarning: pkg_resources is deprecated as an API
. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources # noqa: TID251
Best run 40a657dd auc=0.7496 params={'model_kind': 'logreg', 'C': '1.0', 'data_fingerprint': 'fb2752fe23'}
Registered model 'cartvista-churn' already exists. Creating a new version of this model...
Created version '3' of model 'cartvista-churn'.
Registered cartvista-churn v3 and set alias @champion
```

Figure 5. Terminal output from `register_model.py` showing the registered model version and `@champion` alias.

8. Consumer Program Output

The consumer program loads the registered model through the `cartvista-churn@champion` alias without importing the training code. This demonstrates the intended decoupling between model training and downstream model consumption.

```
(.venv) PS C:\Users\Dell\OneDrive\Desktop\cartvista-mlops> python consume_model.py
import pkg_resources # noqa: TID251
customer_000 -> churn prediction: 0
customer_001 -> churn prediction: 0
customer_002 -> churn prediction: 0
customer_003 -> churn prediction: 0
customer_004 -> churn prediction: 0
```

Figure 6. Terminal output from `consume_model.py` showing five customer churn predictions.

9. Reflection – Separate MLflow Instances

If Priya and Arjun each run MLflow independently on their own laptops, their tracking stores and registries are separate. There would be no single shared source of truth for experiments, model versions or the champion alias, making it difficult for the team to know which model is current and to reproduce or compare another person's run. A shared MLflow tracking and registry backend is needed for collaborative, centralized

experiment and model management.

10. Submission Checklist

Required item	Status
Experiment table / MLflow comparison screenshot	Included
Models / registry and @champion evidence	Included
consume_model.py terminal output	Included
Step 2 checkpoint	Included
E1 – Precision and Recall	Included
E3 – Tags	Included
E4 – Data Fingerprint	Included
Reflection paragraph	Included
E5 – Autolog vs manual	Optional / not required